

cs-mail Express Lanes — Implementation Plan

Recipient-Granted, Bond-Free Contact for Chosen Organizations

C-SQD

Draft 0.2 — August 2026

Nonnormative implementation plan. This document records settled design decisions and a build order for *express lanes*, the launch-feature form of the scoped-capability extension that the *cs-mail Protocol Specification* marks deferred. It extends the *Rust Reference Architecture* and is written against the current Rust workspace. It does not change the settlement or privacy semantics of the specification; where the two differ, the specification controls. The user-facing explanation of the feature is the companion memo, *Express Lanes for Transactional Mail*.

1 The Central Observation

The money kernel, `cs-mail-protocol::transition`, runs only when value moves. `IssueContactTerms` already returns `TermsOutcome::NoBondRequired` for an accepted relationship, but that quotation result is not itself delivery. The shared bond-free admission path therefore lives beside the kernel: it authenticates the sender, locks the directed relationship, validates durable encrypted content, verifies either acceptance or a lane, and writes the delivery outbox intent. An express lane is a sibling of accepted-sender admission, not a new settlement transition.

This single fact determines the shape of the whole feature: express lanes are implemented as a permission layer consulted *beside* the kernel, and the deterministic settlement kernel is left unchanged.

2 Settled Design Decisions

1. **Edge subsystem, not kernel state.** A lane is evaluated at the admission edge, before the kernel is consulted—mirroring how accepted delivery already avoids the kernel. It follows the bounded, idempotent, in-memory “book” pattern already used by `FundingBook` and `FederationBook` in `cs-mail-adapters`, with a durable PostgreSQL coordinator for production-shaped admission.
2. **Domain as identity, for legacy senders.** A DMARC-authenticated sending domain resolves to a single synthetic sender protocol identity, so every authenticated address at that domain shares one lane and the per-pair aggregate is preserved.
3. **One lane type, a subject enum.** A lane’s subject is either a native opaque handle or a recipient-granted legacy domain scope. The handle identifies the pair; a separate recipient signature authorizes the complete grant. The same versioned record’s subject can be swapped on adoption.

4. **DMARC verification as evidence-as-data.** Verification happens at the edge and produces a `VerifiedDomain` value. A separately versioned binding joins that proof to the recipient-granted lane domain, so lane evaluation remains deterministic and testable without DNS.
5. **Lifetime on the existing scheduler.** Lanes are time-bound on a sliding window anchored to last activity, bounded by the grant’s hard `not_after`, using the shared `ScheduleTask` infrastructure, `cs_schedules`, and the `cs-mail-worker` lease.
6. **Block and lane are mutually exclusive.** A directed pair is at most one of {blocked, lane-authorized, neither}. Blocking hard-revokes any lane; granting a lane to a blocked pair requires an explicit unblock; unblocking leaves the pair bonded, not re-lane’d.
7. **Issuance is recipient-initiated only.** Lanes are granted at address handover (checkout, signup, booking) or by replying to an organization first. The “accept for receipts only” first-contact path is removed.
8. **Volume is enforced, purpose is disclosed.** Each lane carries a maximum message count per interval that admission consumes atomically. Purpose remains an honest display label because providers cannot inspect encrypted content.
9. **The settlement transition is untouched.** With the first-contact issuance path removed, no new settlement-bearing command is required. Every change is additive.

3 The Admission Decision

Express lanes add one branch to the answer of a question the system already asks: does this inbound message require a bond? The ordered decision, evaluated for a directed (`sender`, `recipient`) pair at a single canonical boundary, is:

1. if the pair is `Blocked`, refuse (`ContactBlocked`);
2. else if the relationship is `Accepted`, deliver bond-free through the existing accepted-sender path;
3. else if a valid, in-window lane authorizes the pair, deliver bond-free through the lane path;
4. else require a bond and proceed through the kernel exactly as today.

The second and third branches share one durable bond-free admission path. A lane admission produces no bond, moves no ledger value, opens no relationship solicitation episode, and advances no repeated-attempt state—the same properties the specification already requires of a capability-authorized message. Because it never calls `transition`, those properties hold by construction rather than by added kernel logic.

```
enum AdmissionOutcome {
    Refused,           // blocked pair
    BondFreeAccepted,  // existing accepted-sender path
    BondFreeLane,      // new: valid express lane
    BondRequired,      // existing kernel settlement path
}
```

For native mail, the sender signs the complete bond-free admission request, including content reference, evidence, idempotency key, protocol version, deployment domain, and intended provider. A legacy SMTP sender cannot supply a native protocol signature: the trusted ingress service instead constructs the request only after its configured `DmarcVerifier` returns passing evidence.

4 The Lane Object and Its Store

A lane is a compact recipient-granted record. It carries no money and references no message content.

```
struct Lane {
    subject: LaneSubject,          // native identity or verified domain
    recipient: ProtocolIdentity,
    purpose_label: PurposeLabel, // disclosed, not content-inspected
    rate_limit: RateLimit,        // enforced atomically at admission
    window: Duration,            // sliding lifetime
    not_after: CanonicalTime,     // signed hard validity bound
    last_activity: CanonicalTime,
    mode: LaneMode,              // Expiring | Persistent
    version: Version,
}

enum LaneSubject {
    Native(ScopedHandle),        // opaque identifier from cs-mail-privacy
    LegacyDomain(DomainIdentity), // recipient-granted domain scope
}
```

The stored lane is created only from a `SignedLaneGrant`. The recipient signature covers the subject, directed protocol pair, purpose label, rate limit, sliding lifetime, hard validity interval, mode, protocol version, deployment, intended provider, operational key, and grant version. `ScopedHandle` is an opaque identifier, not the signature or the authority by itself.

The lane store is a new, compartmentalized access domain: its own database table, credentials, encryption keys, and access role, per the specification’s relationship-graph privacy requirements. It is not a column on the relationship aggregate and not part of `protocol_state`. A ledger or telemetry compromise must not yield the list of organizations a recipient trusts. Grant, renew, and revoke operations are idempotent record transitions with no ledger coupling, exactly like the existing adapter books.

The storage coordinator enforces one invariant under the relationship aggregate lock: for any directed pair, at most one of {blocked, lane-authorized} holds at a time.

5 Legacy Senders: Domain as Identity

Most transactional senders will not be on *cs-mail* at launch, so the common case is legacy email authenticated by DKIM and DMARC. The inbound edge verifies DMARC under RFC 9989 and produces a `VerifiedDomain`; lane evaluation consumes that value and never performs DNS or signature work itself. This keeps lane logic pure and unit-testable and confines the messy legacy-world cryptography to one edge boundary.

A passing result keeps the RFC 5322 author domain, independent DKIM and SPF pass facts (each with its applicable strict or relaxed alignment mode), and canonical evaluation time as evidence.

It does not claim that the verifier exposed a DMARC policy-record or organizational domain when the underlying library did not. Lane identity is instead the canonical IDNA-ASCII domain scope deliberately selected by the recipient’s grant; it is never whichever DKIM or SPF identifier happened to pass. The authenticated author must be that domain or a subdomain of it. That lane domain resolves through a versioned, domain-separated mapping to a *single* synthetic sender protocol identity. Consequently `statements@bank.com` and `alerts@bank.com` are one sender to the system, one lane covers both, and admission still keys to a single (`sender`, `recipient`) aggregate. A message that fails DMARC alignment to a lane’s domain is simply not on the lane, regardless of its display name or local-part. DMARC resists unauthorized use of the expected domain; the separate expected-domain match rejects look-alike domains.

The workspace now has a dedicated `cs-mail-smtp-gateway` library. It pins Stalwart Labs’ `mail-auth` 0.12.1 with default features disabled and the minimal Ring/Hickory DNS feature set. The gateway accepts the remote IP, EHLO identity, MAIL FROM (including the null reverse-path case), receiver hostname, raw message, and canonical receipt time. It bounds message size, header size and count, DKIM-signature count, and receipt/system-clock skew before performing DNS-backed verification. Temporary DNS failures remain distinguishable from authentication failure, malformed input, and resource exhaustion for SMTP policy.

6 Native Senders: Pairwise Handles

For a sender already on *cs-mail*, the lane binds to the pairwise-scoped `ScopedHandle` already provided by `cs-mail-privacy`. The recipient signs the full lane grant containing that handle. The provider verifies the grant at admission without learning which organization it authorizes, so the native lane is fully private where the legacy lane must expose a domain. When a legacy sender adopts *cs-mail*, its lane’s subject is swapped from `LegacyDomain` to `Native`, upgrading the lane in place and preserving its history.

Native identity also enables finer control that domain granularity cannot offer. Because a legacy domain is a single identity, block and lane are mutually exclusive at domain granularity—one cannot block a bank’s marketing while keeping a lane for its fraud alerts. A native organization that uses distinct protocol identities for distinct purposes can be blocked on one and lane-granted on another.

7 Lifetime and Renewal

A lane’s clock is its last authenticated message, not its issue time. Each admitted lane message bumps `last_activity` and reschedules the lane’s horizon; a lane in active use therefore never lapses on a timer, while a dormant one drifts toward expiry.

Two modes cover the range the memo describes:

- **Expiring** (default): at the horizon the recipient receives a gentle re-confirm prompt rather than a silent deletion. The window is set per sender.
- **Persistent**: the same renew-on-activity behavior, but the horizon keeps the lane active and emits a review reminder instead of lapsing it. The signed `not_after` remains a hard stop. This is the mode for a bank’s fraud alerts or a hospital’s results, where a silently lapsed lane would be costly.

Both modes reuse the leased `cs_schedules` plus `cs-mail-worker` machinery: a scheduled task fires at `last_activity + window` and either lapses the lane or emits a review effect through the

outbox. The generic `ScheduleTask` and `ScheduleChange` types live in `cs-mail-primitives`; no second lease substrate is required.

8 Block and Lane Interaction

Block and lane are opposite ends of one recipient control, so the model treats them as mutually exclusive rather than as independent flags. The two directions are deliberately asymmetric:

- **Blocking hard-revokes the lane, unconditionally.** Block is the most restrictive state and a deliberate protective action, so it tears down any lane in the same commit that records the block. After a later unblock the organization does *not* regain its lane; it returns to a bonded stranger, matching the kernel’s existing `unblock` semantics.
- **Granting a lane never silently unblocks.** Because issuance is one-tap or automatic, an issuance flow must not silently defeat a block the recipient deliberately set. A grant targeting a blocked pair surfaces the block and requires an explicit unblock-and-allow confirmation.

This mutual-exclusion invariant also resolves the coordination question raised by placing lanes outside the kernel. Block state lives in the relationship aggregate; the lane lives in the edge store. Rather than reconciling two independent truths, blocking is one state transition that revokes the lane in the same commit, and lane admission reads the pair’s block state transactionally—by taking the relationship aggregate’s row lock, or by consulting a minimal transactional block flag—so a block that commits first is always observed. Domain-as-identity is what makes “the pair” well defined: blocking `bank.com` and holding a lane for `bank.com` concern the same directed identity pair.

The reference implementation chooses the aggregate row lock. A future split-store deployment must preserve the same linearization contract through a narrowly authorized transactional coordinator.

9 Issuance

A lane is granted only by a deliberate recipient act:

- **At address handover** (checkout, signup, booking). The client records the web origin it is on as the expected sending domain, then completes the grant on the first DMARC-authenticated message from that domain, showing a one-tap confirmation that names the *authenticated* domain rather than the display name. Seeding the expectation from the origin turns “which sender?” into a match and lets the recipient catch a look-alike domain at grant time.
- **On replying first.** Writing to an organization auto-opens a return lane, since the recipient has plainly invited a reply.

The first-contact “accept for receipts only” path is removed. A cold sender uses the ordinary bonded gate; it can be accepted as a full standing contact, rejected, or left to expire, but it is not granted a lane from that interaction. The consequence is minor and recoverable: an organization one would have preferred to keep on a narrow lane must instead be reached through handover or a reply, or accepted fully. The benefit is that issuance touches no money and no kernel state.

10 Changes by Crate

- **cs-mail-capabilities** holds the lane “book”: the lane record, subject enum, and idempotent grant/renew/revoke transitions, with no ledger coupling and the block/lane mutual-exclusion invariant.
- **cs-mail-adapters** defines the async, library-neutral SMTP authentication request, typed failures, and versioned evidence model, alongside the existing outbound downgrade route.
- **cs-mail-smtp-gateway** implements that boundary with pinned Stalwart **mail-auth**, minimal crypto/DNS features, resource limits, and receipt-time skew checks.
- **cs-mail-storage-postgres** gains a migration for the compartmentalized lane table under its own access role and keys, plus the transactional block/lane coordination.
- **cs-mail-primitives**, **cs-mail-worker**, and **cs_schedules** supply shared task types and the existing lease for lane horizon tasks (lapse or review); no new scheduling substrate.
- **cs-mail-privacy** supplies **ScopedHandle** as the native pair identifier; **cs-mail-capabilities** supplies the separately signed authority.
- **Service and capability layers** add authenticated ingress and domain-separated, versioned signing representations for grants and bond-free admission.
- **cs-mail-protocol::transition** (the settlement kernel) is unchanged. Shared effect and scheduling representations grow additively.

11 Build Order

Each stage is independently testable and none requires kernel work.

1. **Native lanes, end to end (implemented).** The lane book, the admission branch, grant and revoke, and the block/lane mutual-exclusion invariant with its transactional coordination established the permission layer and its hardest correctness property first.
2. **Lifetime and renewal (implemented).** Sliding window, the two modes, and the scheduled lapse/re-confirm tasks on the existing worker lease.
3. **Legacy senders (implemented).** The inbound DMARC verifier, domain-to-identity resolution, and the legacy subject, with independent SPF and DKIM evidence and expected-domain mismatch enforcement.
4. **Issuance flows.** Address-handover with web-origin seeding and confirm-on-first-authenticated-message, and reply-first auto-grant.

12 Open Questions and Risks

- **Split-store linearization.** The reference implementation uses the relationship aggregate row lock. Any future split-store deployment must preserve that ordering contract with a narrowly authorized coordinator.

- **Renewal signal.** An authenticated, admitted lane message renews the sliding horizon using monotonic canonical time. Decide separately whether recipient reads or replies should count.
- **DMARC operations.** The reference selects pinned Stalwart `mail-auth` and keeps DNS out of lane logic. A deployment must still operate resolver policy, map typed failures to SMTP replies, monitor latency, and keep its system clock within the configured receipt-time skew. Senders without a DMARC pass receive no legacy lane; a weaker grant would be a separately named and reviewed feature.
- **Per-subject retention.** Native lane subjects are opaque; legacy subjects expose a domain. Retention and audit rules differ per subject and must be encoded in the store’s retention classes.
- **Issuance as a surface.** Rate-limit issuance confirmations and prevent the lane store or its prompts from becoming an enumeration oracle.

13 Documentation Status Changes

Shipping express lanes promotes scoped capabilities from a deferred optional extension to a launch feature. This is a status change across the whitepaper, specification, reference architecture, and deployment profile, not a change to their settlement or privacy meaning. The specification’s existing capability rules—no solicitation, no repeated-attempt advancement, block overrides a capability—are satisfied here by construction, because a lane-authorized message never enters the settlement kernel. The companion memo remains the user-facing explanation of the feature.